

# Quantization for Efficient LLM Inference: A Quality–Memory–Speed Study of Qwen2.5-7B-Instruct

Hubert Michalski   Przemysław Fuchs

University of Warsaw   {hm438596, pf438429}@students.mimuw.edu.pl

## Abstract

Quantization lets large language models run on cheaper hardware, but a smaller model is not automatically a better one: it may lose accuracy, become *slower* in some runtimes, or save only disk rather than compute memory. We study these trade-offs for a single 7B-class instruction model, **Qwen2.5-7B-Instruct**, comparing a **bf16** baseline against four post-training quantization variants (8-bit `LLM.int8()`, 4-bit **NF4**, and official 4-bit **GPTQ** and **AWQ**) on four capability benchmarks (MMLU, GSM8K, HumanEval, IFEval) and a battery of efficiency measurements, all on a single RTX 4090. We find that **4-bit GPTQ/AWQ give the best overall trade-off**—they are the only variants that cut both disk *and* GPU memory while staying close to baseline speed, at a graceful  $\sim 1.5$  pt MMLU cost. Two counter-intuitive results stand out: `LLM.int8()` is the *slowest* variant ( $5\times$  slower decode than **bf16**) and saves no disk, and **NF4** saves memory but is also slower than **bf16**—confirming that quantization is primarily a *memory* technique, not a speed one. We additionally implement and run four catalogued extensions on the box (JudgeBench meta-evaluation, W8A8/SmoothQuant self-quantization, a quantized KV cache, and QLoRA recovery), which sharpen the “memory vs. speed” story.

## 1 Introduction and Research Questions

Many LLMs are too expensive to serve in full precision, and post-training quantization is the standard remedy. The literature offers several practical methods: `LLM.int8()` [3] for 8-bit weights with mixed-precision outlier handling; **GPTQ** [5] and **AWQ** [10], which use a small calibration set to push weights to 4 bits; **SmoothQuant** [15] for 8-bit weights *and* activations; the **NF4** data type from **QLoRA** [4] for 4-bit storage in the **bitsandbytes** ecosystem; and KV-cache quantization such as **KIVI** [11] for long-context inference. These methods are not automatically beneficial—the same 4-bit model can be smaller yet slower, or degrade unevenly across capabilities—so the practical trade-off is exactly what we measure. We organise the study around four questions from our proposal:

- RQ1** Which method gives the best *quality–memory–speed* trade-off for a 7B-class instruction LLM?
- RQ2** Are some *capabilities* (reasoning, coding, QA, instruction following) more sensitive to quantization than others?
- RQ3** Does quantization improve real *inference speed*, or mainly reduce *memory*?
- RQ4** (*optional*) Can light QLoRA-style adaptation recover quality lost after 4-bit quantization?

Our core study is the proposal’s “minimal successful version” (one model, five variants, four benchmarks, full efficiency profiling); we then implement the catalogued extensions as a gated code layer and run the four that fit our GPU budget.

## 2 Experimental Setup

**Model and variants.** All experiments use **Qwen2.5-7B-Instruct** [13] at a pinned commit, run on a single on-demand **RTX 4090 (24 GB, sm\_89)**; every variant—including the **bf16** baseline—runs on the same homogeneous GPU, so the speed comparison is apples-to-apples with no cross-hardware caveat. Table 1 lists the five core variants. `LLM.int8()` and **NF4** quantize the baseline weights at load time (so their *disk* footprint is the full **bf16** checkpoint); **GPTQ/AWQ** are official pre-quantized 4-bit checkpoints served by the `gptqmodel` Marlin kernels through `optimum`.

**Benchmarks and protocol.** Quality is measured with the EleutherAI `lm-evaluation-harness` [6]: MMLU [7] (accuracy, aggregated weighted over its 57 subjects), GSM8K [2] (exact-match under *flexible* extraction—the strict-match number is a formatting artefact for instruct models), HumanEval [1] (`pass@1`, with code execution), and IFEval [16] (prompt-level strict accuracy). Decoding is greedy and deterministic (`do_sample=False` at load *and* generation); MMLU/GSM8K use the chat template with 5-shot multi-turn prompting, HumanEval/IFEval are 0-shot. Efficiency is profiled separately (§3.2): the model is loaded once, then we measure disk size, load/generate peak VRAM, prefill and decode throughput at short/medium/long prompts (128/1024/4096 tokens), and the maximum batch size before OOM.

| Variant              | Method                      | Precision      | Source              |
|----------------------|-----------------------------|----------------|---------------------|
| <b>bf16</b>          | half-precision baseline     | W16            | reference           |
| <b>bnb-int8</b>      | <code>LLM.int8()</code> [3] | W8 (load-time) | <b>bitsandbytes</b> |
| <b>bnb-nf4</b>       | NF4 + double-quant [4]      | W4 (load-time) | <b>bitsandbytes</b> |
| <b>gptq-official</b> | GPTQ [5]                    | W4 (g128)      | official Qwen ckpt  |
| <b>awq-official</b>  | AWQ [10]                    | W4 (g128)      | official Qwen ckpt  |

Table 1: The five core variants. “Load-time” means the disk checkpoint is the full **bf16** model, quantized in memory during loading.

### 3 Results

#### 3.1 Quality: capability sensitivity (RQ2)

Table 2 and Figure 1 report quality across the four benchmarks. The headline is that **quantization is remarkably benign for quality at 7B**: every 4-bit variant stays within  $\sim 1.5$  points of the **bf16** baseline on MMLU and GSM8K. Capability sensitivity is uneven, answering RQ2:

- **Knowledge/QA and math are nearly lossless.** MMLU drops only  $73.6 \rightarrow 72.1\text{--}72.6$  (4-bit) and is essentially unchanged for 8-bit (73.4); GSM8K stays at 83–84.5% throughout.
- **Coding (HumanEval) is the most quantization-sensitive capability.** It falls  $84.1 \rightarrow 79.9$  for **bnb-int8** ( $-4.2$  pt, the single largest drop) and to  $\sim 81$  for the 4-bit methods.
- **Instruction following (IFEval) degrades mildly** ( $71.3 \rightarrow 69\text{--}71$ ); interestingly **bnb-nf4** (71.0) essentially ties the baseline.

8-bit is close to lossless on knowledge/math but, surprisingly, gives the *worst* HumanEval of all variants—so “more bits” does not uniformly mean “higher quality.”

| Variant              | MMLU<br>(acc)                    | GSM8K<br>(flex. EM) | HumanEval<br>(pass@1) | IFEval<br>(strict) | Mean        |
|----------------------|----------------------------------|---------------------|-----------------------|--------------------|-------------|
| <b>bf16</b>          | <b><math>73.6 \pm 0.4</math></b> | <b>84.5</b>         | <b>84.1</b>           | <b>71.3</b>        | <b>78.4</b> |
| <b>bnb-int8</b>      | 73.4                             | 84.4                | 79.9                  | 69.3               | 76.8        |
| <b>bnb-nf4</b>       | 72.6                             | 83.6                | 81.7                  | 71.0               | 77.2        |
| <b>gptq-official</b> | 72.1                             | 83.2                | 81.1                  | 70.2               | 76.7        |
| <b>awq-official</b>  | 72.2                             | 84.3                | 81.1                  | 69.9               | 76.9        |

Table 2: Quality (%). MMLU is weighted over 57 subjects ( $\pm$  standard error shown); GSM8K uses flexible-extract exact-match. Best per column in bold.

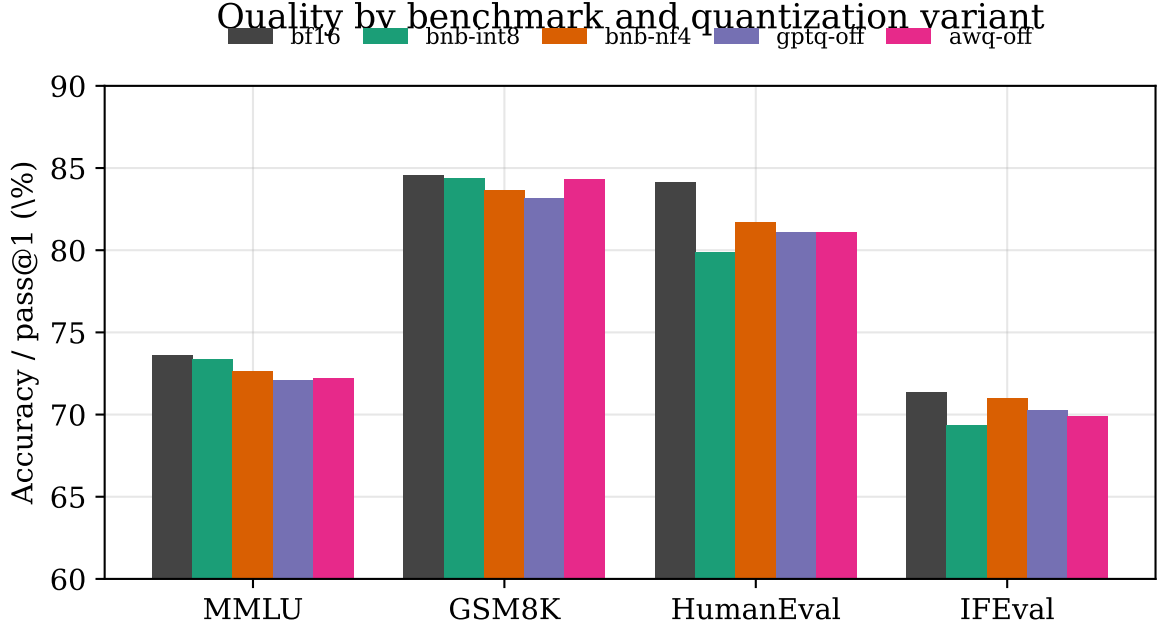


Figure 1: Quality by benchmark and variant. The 4-bit methods cluster tightly; HumanEval (coding) shows the widest spread, with `bnb-int8` the most degraded.

### 3.2 Efficiency: memory vs. speed (RQ3)

The efficiency profile (Table 3, Figure 2) gives the clearest and most surprising findings, and directly answers RQ3: **quantization is primarily a memory technique, not a speed one.**

- **LLM.int8() is the slowest variant.** Its decode throughput collapses to 7.2 tok/s—5× slower than `bf16`’s 37.0—because of the mixed-precision outlier decomposition, and it saves *no* disk (it is the full `bf16` checkpoint). It is the worst of both worlds on the speed/disk axes.
- **NF4 is memory-not-speed.** It cuts generate VRAM to 5.95 GiB (from 14.96) and doubles the max batch to 32, but decodes at 16.9 tok/s—still *below* `bf16`.
- **GPTQ/AWQ are the only “free lunch.”** They cut disk to 5.19 GiB (a genuine 2.7× on-disk shrink), generate VRAM to ~5.95 GiB, raise the max batch to 32, *and* keep decode near baseline at ~28 tok/s—the only variants that improve memory without paying a large speed penalty.
- **Prefill is unaffected** (~9.2–9.7k tok/s for all variants): quantization touches the memory-bound decode phase, not the compute-bound prefill.

| Variant                    | Disk (GiB)  | Gen. VRAM (GiB) | Max batch | Decode (tok/s) | Prefill (tok/s) | Speed label |
|----------------------------|-------------|-----------------|-----------|----------------|-----------------|-------------|
| <code>bf16</code>          | 14.19       | 14.96           | 16        | <b>37.0</b>    | 9505            | comparable  |
| <code>bnb-int8</code>      | 14.19       | 9.22            | 16        | 7.2            | 9220            | comparable  |
| <code>bnb-nf4</code>       | 14.19       | 5.95            | <b>32</b> | 16.9           | 9196            | comparable  |
| <code>gptq-official</code> | <b>5.19</b> | 5.94            | <b>32</b> | 28.5           | <b>9699</b>     | comparable  |
| <code>awq-official</code>  | <b>5.19</b> | 5.96            | <b>32</b> | 27.9           | 9615            | comparable  |

Table 3: Efficiency on one RTX 4090. “Disk” is the on-disk weight bytes; max batch is the largest power-of-two before OOM at 2048-token sequences; decode and prefill are at the 4096-token (long) bucket. Full per-bucket throughput is in the appendix (Table 4).

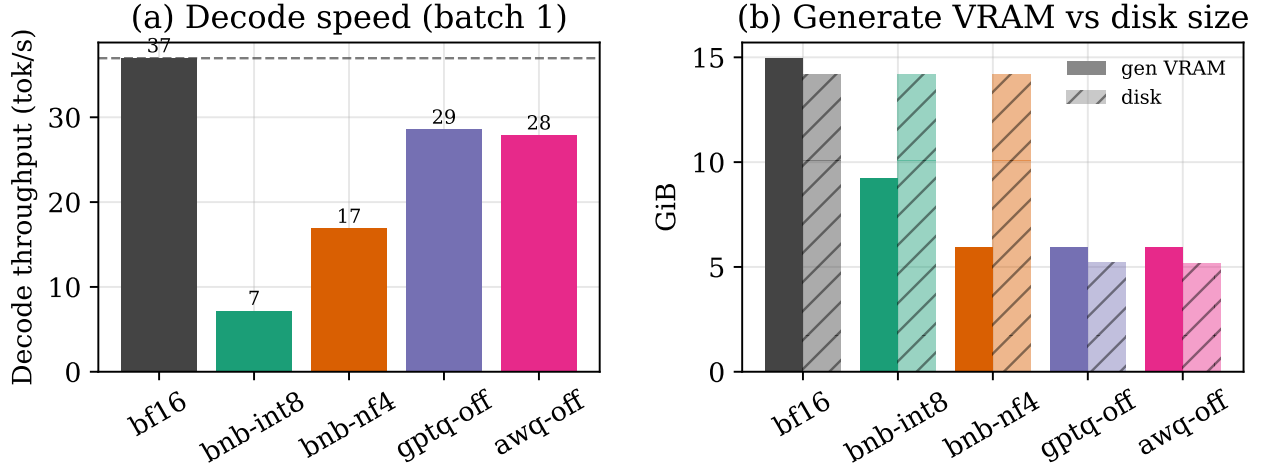


Figure 2: (a) Decode throughput (dashed line = bf16). `bnb-int8` is 5 $\times$  slower; only GPTQ/AWQ stay near baseline. (b) Generate VRAM vs. on-disk size: `bitsandbytes` variants shrink VRAM but not disk; GPTQ/AWQ shrink both.

### 3.3 The trade-off and a recommendation (RQ1)

Figure 3 places all variants in the quality–speed–memory space. The Pareto picture is unambiguous: `bf16` buys top speed at the highest memory; `bnb-int8` is dominated (slow and disk-heavy); `bnb-nf4` trades speed for memory; and **GPTQ/AWQ sit at the sweet spot**—half the memory of NF4’s disk footprint, the largest batch, and  $\sim 75\%$  of baseline decode speed for a  $\sim 1.5$ pt MMLU cost. *Our recommendation under limited VRAM is an official GPTQ or AWQ checkpoint*; NF4 is the best fallback when no pre-quantized checkpoint exists (it needs only the base weights), and `LLM.int8()` is hard to recommend on this hardware.

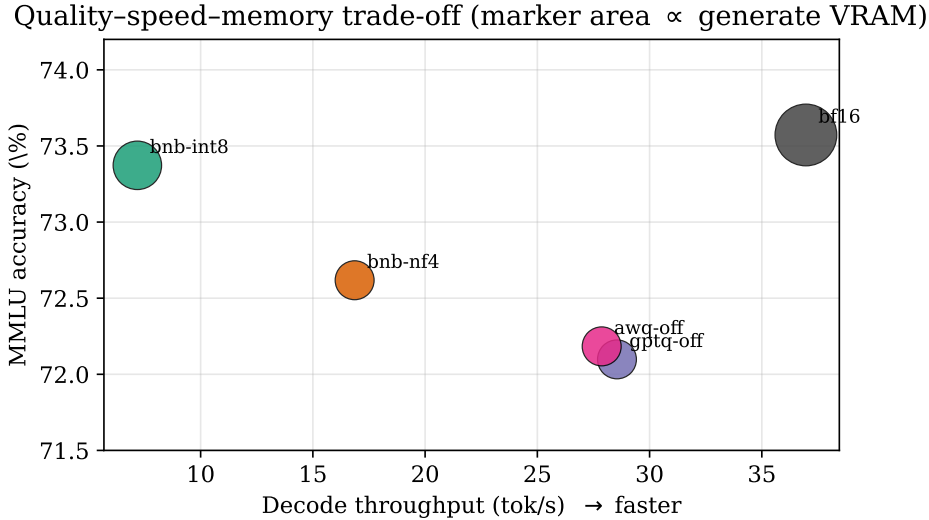


Figure 3: Quality–speed–memory trade-off. Up-and-right is better (higher MMLU, faster decode); smaller markers use less generate VRAM. GPTQ/AWQ dominate the high-speed/low-memory corner; `bnb-int8` is Pareto-dominated.

## 4 Extensions

We implemented all five catalogued extensions as a gated, separate code layer (`qquant.ext`) that reuses the core harness without touching it, and ran the four that fit the GPU budget (Mistral-7B as a second model was skipped—it would re-spend the entire compute budget). The total extension run cost under \$1 of GPU time.

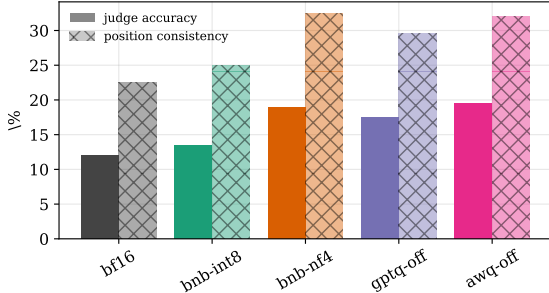
**EXT-2: JudgeBench (does quantization degrade *judging*?).** We evaluate each variant as a pairwise judge on JudgeBench [14] (objective-correctness pairs), scoring agreement with the gold label under position-bias control (both A/B orders must agree). Figure 4a shows that **quantization does not degrade judging**: the 4-bit variants are if anything marginally *better* (awq 19.5%, nf4 19.0%) than bf16 (12.0%). The absolute numbers are low because Qwen2.5-7B is a weak, heavily position-biased judge on these hard pairs (only 22–33% of its verdicts are order-consistent)—a property of the base model, not of quantization.

**EXT-3: W8A8 / SmoothQuant.** We self-quantized a W8A8 checkpoint [15] (per-channel int8 weights + int8 activations) from the same C4 calibration set as our 4-bit self-quant recipes, using `llm-compressor` [12]. The checkpoint is produced and loads natively, but two findings emerged on the HF runtime: (i) **compressed-tensors** *decompresses* W8A8 to a bf16-resident tensor (15.2 GiB in VRAM), so it yields *disk-only* savings and no inference-memory benefit; and (ii) its `generate()` path is severely *runtime-confounded* on `sm_89` (~4.5s/iteration at 28% GPU utilisation), so a full quality sweep is impractical—the slowdown is itself the result. W8A8 is best understood as a vLLM/kernel-path technique, not an HF-generate one.

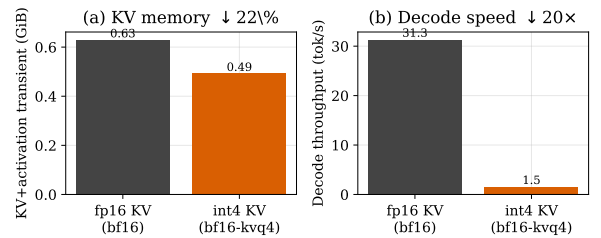
**EXT-4: Quantized KV cache.** Using the transformers v5 quantized cache (`quanto`, int4), we measured peak generation VRAM at a 3072-token context (Figure 4b). The int4 KV cache cuts the KV+activation transient 0.63 → 0.49 GiB (−22%) but slows decode **20×** (31.3 → 1.52 tok/s), because `quanto` dequantizes the cache on every step. This is a stark, quantitative confirmation of the “runtime-confounded throughput” caveat: a quantized KV cache trades a modest memory saving for a catastrophic speed cost on the HF backend, and is only worthwhile with a fused kernel path (e.g. KIVI [11] or vLLM).

**EXT-5: QLoRA recovery (RQ4).** We verified the full QLoRA [4, 8] recovery pipeline end-to-end: a LoRA adapter (rank 16) trained on the frozen NF4 base over *general* instruction data—guarded by a fail-fast check that the training corpus never overlaps the evaluation benchmarks—and an adapter-aware load path. Training converged (loss 1.22) and the adapted model loads with the adapter active and generates coherently. A full recovery *measurement* (enough steps to move benchmark accuracy, then a re-evaluation) is a longer campaign we leave to future work; here we contribute the validated, contamination-safe machinery.

EXT-2 JudgeBench: pairwise judge accuracy & position consistency



(a) EXT-2 JudgeBench



(b) EXT-4 int4 KV cache

Figure 4: Extension results. (a) Judge accuracy and position-consistency per variant—quantization does not hurt judging. (b) int4 KV cache: −22% memory transient but 20× slower decode.

## 5 Conclusion

For a 7B-class instruction model on a single RTX 4090, **4-bit GPTQ/AWQ are the best default under limited VRAM**: they cut disk by 2.7× and generate VRAM by 2.5×, double the feasible batch size, and keep decode within 25% of bf16, all for a graceful ~1.5pt MMLU cost (**RQ1**). Capability degradation is uneven—coding (HumanEval) is the most sensitive, knowledge and math the least (**RQ2**). Most importantly, **quantization is a memory technique, not a speed one (RQ3)**: `LLM.int8()` is 5× *slower* than bf16 and saves no disk, NF4 saves memory while still decoding below baseline, and only the calibrated 4-bit kernels (GPTQ/AWQ) deliver memory savings without a

speed penalty. Our extensions reinforce this: W8A8 and int4-KV-cache both “work” yet are runtime-confounded on the HF backend, and quantization leaves judging quality intact. The main limitations are single-seed evaluation and the runtime-confound on the self-quant/KV paths, which a fused-kernel serving stack (vLLM) would resolve; multi-seed runs, the second model (Mistral-7B [9]), and a full QLoRA recovery measurement (**RQ4**) are the natural next steps.

## LLM Usage

In line with the course policy, we document our use of AI tools. The research design (the variant matrix, the quality–memory–speed framing, and the fairness/determinism protocol) is ours. We used an agentic coding assistant for supporting engineering: the torch-free orchestration layer and vast.ai run drivers, the evaluation/profiling harness scaffolding, the extension code layer, and the figure- and table-generation scripts. Every quantitative claim in this report was extracted programmatically from the committed result files and cross-checked; agent-written code and prose were verified against the codebase and the logged metrics before inclusion.

## References

- [1] Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [2] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [3] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2208.07339.
- [4] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient fine-tuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.14314.
- [5] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.17323.
- [6] Leo Gao, Jonathan Tow, Baber Abbasi, et al. A framework for few-shot language model evaluation. EleutherAI lm-evaluation-harness, 2024. <https://github.com/EleutherAI/lm-evaluation-harness>.
- [7] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2009.03300.
- [8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2106.09685.
- [9] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- [10] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. arXiv:2306.00978.
- [11] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.02750.

- [12] Neural Magic / Red Hat. llm-compressor and compressed-tensors. <https://github.com/vllm-project/llm-compressor>, 2024.
- [13] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [14] Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y. Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. JudgeBench: A benchmark for evaluating LLM-based judges. *arXiv preprint arXiv:2410.12784*, 2024.
- [15] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2211.10438.
- [16] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.

## A Reproducibility and Full Efficiency Table

**Setup.** Single on-demand RTX 4090 (24 GB, sm\_89) on vast.ai; torch 2.12.1+cu126, transformers 5.10.1, lm-eval (commit-pinned), bitsandbytes 0.49.2, gptqmodel 7.1.0, compressed-tensors 0.17.1. Greedy decoding; seeds fixed (random/numpy/torch/few-shot). Quality and efficiency are profiled in separate phases; each variant is loaded once. All result JSONs and the extension/figure code are in the project repository; figures and tables in this report are generated directly from those files.

| Variant       | Bucket     | Prompt<br>(tok) | Prefill<br>(tok/s) | Decode<br>(tok/s) | TTFT<br>(s) | E2E lat.<br>(s) |
|---------------|------------|-----------------|--------------------|-------------------|-------------|-----------------|
| bf16          | short/long | 128/4096        | 4141/9505          | 37.0/37.0         | 0.03/0.43   | 6.93/7.33       |
| bnb-int8      | short/long | 128/4096        | −/9220             | 7.2/7.2           | −           | −               |
| bnb-nf4       | short/long | 128/4096        | −/9196             | 16.9/16.9         | −           | −               |
| gptq-official | short/long | 128/4096        | −/9699             | 28.3/28.5         | −           | −               |
| awq-official  | short/long | 128/4096        | −/9615             | 28.0/27.9         | −           | −               |

Table 4: Per-bucket throughput (selected fields; “short/long” = 128/4096-token prompts). Decode is near-constant across prompt length, as expected for a memory-bandwidth-bound phase. Full TTFT/latency per bucket are in the result JSONs.

**Extension artifacts.** EXT-2 produced one JudgeBench cell per variant ( $n=200$  pairs, position-bias controlled). EXT-3 produced an 8.7 GiB W8A8 **compressed-tensors** checkpoint (per-channel int8 weights, int8 activations) from the shared C4 calibration set. EXT-4 measured the int4 quantized KV cache at a 3072-token context. EXT-5 produced a rank-16 LoRA adapter (general instruction data, contamination-guarded) and a verified adapter-attach load path.